

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	6-Pthreads并行构造	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.5.7

实验目的

- 掌握使用Pthreads分解for循环的方法
- 掌握动态链接库的生成和使用
- 使用自己构造的并行结构改造不同的openmp并行结构

实验过程与核心代码

构造parallel_for函数

```
typedef struct{
    int start;
    int end;
    int inc;
    void *(*functor)(int, void*);
    void *arg;
    int thread_id;
    int num_threads;
}p_args;
```

```

void parallel_for(int start, int end, int inc, void *(*functor)(int, void*),
void *arg, int num_threads){
    pthread_t *threads = (pthread_t *)malloc(num_threads *
sizeof(pthread_t));
    p_args * args = (p_args *)malloc(num_threads * sizeof(p_args));

    for(int i=0; i<num_threads;i++){
        args[i].start = start;
        args[i].end = end;
        args[i].inc = inc;
        args[i].functor = functor;
        args[i].arg = arg;
        args[i].thread_id = i;
        args[i].num_threads = num_threads;

        pthread_create(&threads[i],NULL,p_func,&args[i]);
    }

    for(int j=0; j<num_threads; j++){
        pthread_join(threads[j],NULL);
    }

    free(threads);
    free(args);
}

```

构造parallel_for函数的基本思路和普通的pthread并行编程相似。使用结构体数组存储需要传递给每个线程的数据，使用pthread_create和pthread_join来创建和合并线程。

```

void* p_func(void* arg){
    p_args *args = (p_args* )arg;
    for(int i = args->start + args->thread_id; i<args->end; i += args->inc){
        args->functor(i, args->arg);
    }
    return NULL;
}

```

线程函数p_func，给每个线程分配一部分循环任务。这里的分配逻辑是，假如有9个任务和3个线程，这三个线程分别执行任务1、4、7，任务2、5、8和任务3、6、9。

应用于矩阵乘法

```

extern void parallel_for(int start, int end, int inc, void *(*functor)(int,
void*), void *arg, int num_threads);

```

```

void* matrix_multiply1(int i,void* arg){
    MatrixArg *mtxarg = (MatrixArg*)arg;
    int *A = mtxarg->A;
    int *B = mtxarg->B;
    int *C = mtxarg->C;
    for(int j=0; j<K; j++){
        C[i * K + j] = 0;
        for(int a=0; a<N;a++){
            C[i*K+j] += A[i*N+a] * B[a*K+j];
        }
    }
}

```

```

MatrixArg mtxarg = {.A = A, .B = B, .C = C1};
parallel_for(0,M,1,matrix_multiply1, &mtxarg,thread_num);

```

在main函数中，调用parallel_for来并行分解最外层的循环，内层循环不变。

在使用动态链接库时，不仅需要在命令行中写出，还需要在代码中使用extern关键字引用对应的函数。

应用于heated_plate_omp

```
parallel_for(1, M - 1, 1, set_w_i, w, num_threads);
parallel_for(0, N, 1, set_w_j, w, num_threads);
```

```
void* set_w_i(int i, void* arg){
    double (*w)[N] = (double (*)[N]) arg;
    w[i][0] = 100.0;
    w[i][N-1] = 100.0;
}
```

在heated_plate_omp中，有多种并行结构。第一种也是最简单的一种结构，只需要传递一个参数，只有一层循环。这种情况下，直接调用并传递参数即可。

```
MeanArg marg = {.w = w, .mean = &mean};

parallel_for(1, M - 1, 1, add_mean, &marg, num_threads);
parallel_for(0, N, 1, add_mean, &marg, num_threads);

mean = mean / ( double ) ( 2 * M + 2 * N - 4 );
printf ( "\n" );
printf ( " MEAN = %f\n", mean );

parallel_for(1, M - 1, 1, set_mean, &marg, num_threads);

void* set_mean(int i, void* arg){
    MeanArg *marg = (MeanArg*)arg;
    double mean = *(marg->mean);
    double (*w)[N] = marg->w;
    for (int j = 1; j < N - 1; j++) {
        w[i][j] = mean;
    }
    return NULL;
}
```

第二种情况，需要传递多个参数，有多层循环。这种情况下，需要将参数组成结构体传输，且只能分解最外层的循环。

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
DiffArg darg = {.w = w, .u = u, .diff = &diff, .mutex = &mutex};
parallel_for(1, M - 1, 1, diff_cmp, &darg, num_threads);
pthread_mutex_destroy(&mutex);
```

```

void* diff_cmp(int i, void* arg) {
    DiffArg *darg = (DiffArg*)arg;
    double (*u)[N] = darg->u;
    double (*w)[N] = darg->w;
    double *diff = darg->diff;
    double my_diff = 0.0;
    for (int j = 1; j < N - 1; j++) {
        if(my_diff < fabs(w[i][j] - u[i][j])){
            my_diff = fabs(w[i][j] - u[i][j]);
        }
    }

    pthread_mutex_t *mutex = darg->mutex;
    pthread_mutex_lock(mutex);
    if (my_diff > *diff) {
        *diff = my_diff;
    }
    pthread_mutex_unlock(mutex);
}

```

第三种也是最复杂的一种情况，涉及到竞态条件保护。首先，需要在main函数中初始化mutex互斥锁，并在结构体中添加这个成员。在diff_cmp函数中，执行到竞态条件保护的部分时，需要加锁来防止其他线程执行这段代码。执行结束后，需要解除互斥锁。

实验结果

矩阵乘法

```

for(int i=0; i<M;i++){
    for(int j=0; j<K; j++){
        if(C[i * K + j] != C1[i * K + j]){
            printf("Incorrect!");
            free(A);
            free(B);
            free(C);
            return 0;
        }
    }
}
printf("Matrix A: %d * %d, Matrix B: %d * %d, Threads: %d, Runtime: %f seconds\n",M,N,N,K,thread_num,runtime);

```

验证与输出部分。

```

paimon@PCLab1:~$ gcc -o lab6-2 lab6-2.c -L. -lpara_for -lpthread -lopenmp
paimon@PCLab1:~$ ./lab6-2 2
Matrix A: 512 * 512, Matrix B: 512 * 512, Threads: 2, Runtime: 0.175894 seconds

```

输出了正常的运行时间，且没有输出“Incorrect”。

heated_plate_openmp

我的代码：

```
paimon@PCLab1:~$ gcc -o heat heat.c -L. -lpara_for -lpthread -fopenmp
paimon@PCLab1:~$ ./heat
```

HEATED_PLATE_OPENMP

C/OpenMP version

A program to solve for the steady state temperature distribution over a rectangular plate.

Spatial grid of 500 by 500 points.

The iteration will be repeated until the change is $\leq 1.000000e-03$

Number of processors available = 2

Number of threads = 2

MEAN = 24.949900

Iteration	Change
-3	
-2-1	1 37.525050
	2 14.071894
	4 6.449618
	8 3.366811
	16 1.685405
	32 0.839828
	64 0.420903
	128 0.210868
	256 0.105516
	512 0.052772

File Edit View Search Terminal Help

MEAN = 24.949900

Iteration	Change
-3	
-2-1	1 37.525050
	2 14.071894
	4 6.449618
	8 3.366811
	16 1.685405
	32 0.839828
	64 0.420903
	128 0.210868
	256 0.105516
	512 0.052772
	1024 0.026396
	2048 0.013196
	4096 0.006599
	8192 0.003299
	16384 0.001650
	27567 0.001000

Error tolerance achieved.

Wallclock time = 58.448650

HEATED_PLATE_OPENMP:

Normal end of execution.

```
paimon@PCLab1:~$
```

输出-3, -2,-1是我在排查段错误问题时添加的输出, 可忽略。

openmp示例代码:

```

HEATED_PLATE_OPENMP
C/OpenMP version
A program to solve for the steady state temperature distribution
over a rectangular plate.

Spatial grid of 500 by 500 points.
The iteration will be repeated until the change is <= 1.000000e-03
Number of processors available = 2
Number of threads =          2

MEAN = 74.949900

Iteration  Change
      1  18.737475
      2   9.368737
      4   4.098823
      8   2.289577
     16   1.136604
     32   0.568201
     64   0.282805
    128   0.141777
    256   0.070808
    512   0.035427
   1024   0.017707
   2048   0.008856
   4096   0.004428

```

```

MEAN = 74.949900

Iteration  Change
      1  18.737475
      2   9.368737
      4   4.098823
      8   2.289577
     16   1.136604
     32   0.568201
     64   0.282805
    128   0.141777
    256   0.070808
    512   0.035427
   1024   0.017707
   2048   0.008856
   4096   0.004428
   8192   0.002210
  16384   0.001043

 16955   0.001000

Error tolerance achieved.
Wallclock time = 11.558981

HEATED_PLATE_OPENMP:
Normal end of execution.

```

示例代码的运行时间更短。

实验感想

本次实验是我个人感觉目前为止难度最高的一次实验。首先，我在动态链接库的生成和使用上遇到了问题。我执行时报错“找不到动态链接库文件”，经过查阅博客，得知是没有将动态链接库所在的文件添加到环境变量中所致。随后，我又遇到了段错误的问题。逐个排查并行模块，发现是函数 `diff_cmp` 中的竞态条件部分没有处理。添加了互斥锁后，问题得到了解决。